3. Tool工具的使用

- 大模型只负责思考，具体的执行一般依赖工具。在 LangChain 1.x 的架构中，工具（Tools）是赋予AI模型“行动能力”的桥梁。如果说大语言模型是应用的“大脑”，负责思考、规划和决策，那么工具就是它的“手和脚”，让它能够执行具体的操作，与现实世界进行交互。

- 工具的三大核心作用：
  - **连接外部世界**：
    - **获取实时信息**：大模型的知识库有其截止日期。工具可以让你查询实时天气、最新新闻、股票价格等动态信息，让模型的回答紧跟时代
    - **调用企业内部 API**：工具能安全地连接公司内部的数据库（如查询客户信息）、业务系统（如创建工单）或知识库，让 AI 助手真正成为业务专家
  - **将“想法”转化为“行动”**：
    - **执行业务操作**：工具可以让 AI 发送邮件、创建日历事件、操作电子表格、在电商平台上下单，实现端到端的自动化流程。
    - **完成精确计算**：当问到复杂的数学问题时，Agent 可以调用计算器工具，而不是依赖模型本身不稳定的“心算”，确保答案的准确性
  - **让 Agent 的决策流程成为可能**：
    - 工具是LangChain 1.x中createAgent执行循环（ReAct 模式：思考 -> 行动 -> 观察）的关键一环。模型先“思考”需要什么信息或执行什么操作，然后通过工具“行动”起来，最后根据工具的返回结果“观察”并生成最终答案。没有工具，Agent 就只是一个高级的“文本生成器”。

3.1. 代理与工具使用的例子

In [1]:
```
from langchain.agents import create_agent
from langchain.chat_models import init_chat_model
from langchain_core.tools import tool

@tool
def get_weather(city: str) -> str:
    """
    获取指定城市的天气信息

    参数:
        city: 城市名称，如"北京"、"上海"

    返回:
        天气信息字符串
    """
    # 模拟天气数据（实际应用中应调用真实API）
    weather_data = {
        "北京": "晴天，温度 15°C，空气质量良好",
        "上海": "多云，温度 18°C，有轻微雾霾",
        "深圳": "阴天，温度 22°C，可能有小雨",
        "成都": "小雨，温度 12°C，湿度较高"
```

```
    }

    return weather_data.get(city, f"抱歉，暂时没有{city}的天气数据")

model = init_chat_model(
    model="qwen3.5:9b",          # 你在 Ollama 中下载的模型名称
    model_provider="ollama",      # 关键参数，指定使用 ollama 提供商
    temperature=0.7,              # 其他参数与使用任何其他模型一样
    base_url="http://localhost:11434"  # 可选，如果你的 Ollama 不是默认地址，则需
)


agent = create_agent(
    model=model,
    tools=[get_weather],  # 只给一个工具
    system_prompt="你是一个帮助助手，可以查询天气信息。"
)

response = agent.invoke({
    "messages": [{"role": "user", "content": "北京今天天气怎么样？"}]
})

print(f"\nAgent 回复：{response['messages'][-1].content}")
```

```
Agent 回复：北京今天的天气是晴天，温度为 15°C，空气质量良好。
```

- 代码说明：
  - 在该例子中使用了代理，代理做明显做了如下几件事情：
    - 调用大模型推理。
    - 根据推理结果调用我们实现的工具。（关键是代理怎么知道调用哪个工具？）

3.2. 实现工具

(1) 编写工具的重点要素

- 编写工具有如下几个重点：
  - 首先，必须使用标注：`@tool`,表明这是一个工具。该工具的作用是：
    - **自动生成工具Schema**：
      - 它会提取函数名作为工具名，函数的文档字符串 (docstring) 作为工具的描述，并根据函数的类型注解生成模型调用所需的参数格式 (JSON Schema)
    - **规范化工具接口**：
      - 装饰后的函数会变成一个标准的 BaseTool对象，支持 .invoke()、.ainvoke() 等统一方法，方便在 LangChain 或 LangGraph 框架中使用
    - **让模型"学会"使用**：
      - 生成的工具可以绑定到支持工具调用 (Tool Calling) 的聊天模型上。模型会根据用户问题，自动判断是否需要调用这个工具以及提取参数

- 其次，基于@tool自动生成Schema，所以必须对工具函数，编写规范的文档注释(docstring)。
  - 工具函数的文档注释是大模型用来理解工具用途，非常重要。
- 然后是对工具函数的参数与返回值注明类型。
  - 参数和返回值的类型，便于大模型处理参数。
- 最后建议：工具函数最好返回字符串，便于大模型理解。

(2) 工具例子

- 考虑工具的参数与返回值：
  - 无参。
  - 单参。
  - 多参。

In [43]:
```
from langchain.tools import tool

@tool
def get_current_time() -> str:
    """获取当前的系统时间。"""
    print("无参数")
    return "2026-04-07"
```

In [44]:
```
from langchain_core.tools import tool
@tool
def control_system(op:str)->str:
    """
    执行对操作系统的控制。
    """
    print(F"参数: op={op}")
    # 调用ctypes完成win + D键操作
    return "已经控制成功"
```

In [45]:
```
from langchain_core.tools import tool
@tool
def calcutor(op:str, a:float, b:float)-> float:
    """
    执行数学运算。
    """
    print(F"参数op={op}, a={a}, b={b}")
    return 100.99
```

(3) 工具的调用

- 工具调用方式有两种：
  - 直接调用。
  - 绑定在模型上调用。
- 工具的调用使用invoke函数（这个需要理解langchain_core.tools.base.BaseTool:langchain_core.tools.structured.StructuredToo
  - 对于无参的工具，也至少传递一个空参数
    - None

- {}
- ()
- []

- 直接调用工具。

In [46]:
```
print(type(get_current_time))
# 直接调用
get_current_time.invoke(None)
get_current_time.invoke({})
get_current_time.invoke(())
get_current_time.invoke([])
```

```
<class 'langchain_core.tools.structured.StructuredTool'>
无参数
无参数
无参数
无参数
```

Out[46]:
```
'2026-04-07'
```

- 绑定模型调用

In [47]:
```
from langchain.chat_models import init_chat_model
model = init_chat_model(
    model="qwen3.5:9b",          # 你在 Ollama 中下载的模型名称
    model_provider="ollama",      # 关键参数，指定使用 ollama 提供商
    temperature=0.7,              # 其他参数与使用任何其他模型一样
    base_url="http://localhost:11434"  # 可选，如果你的 Ollama 不是默认地址，则需
)

model_with_tools = model.bind_tools([get_current_time, control_system, calcutor]
print(model_with_tools)
```

```
bound=ChatOllama(model='qwen3.5:9b', temperature=0.7, base_url='http://localhost:
11434') kwargs={'tools': [{'type': 'function', 'function': {'name': 'get_current_
time', 'description': '获取当前的系统时间。', 'parameters': {'properties': {}, 't
ype': 'object'}}}, {'type': 'function', 'function': {'name': 'control_system', 'd
escription': '执行对操作系统的控制。', 'parameters': {'properties': {'op': {'typ
e': 'string'}}, 'required': ['op'], 'type': 'object'}}}, {'type': 'function', 'fu
nction': {'name': 'calcutor', 'description': '执行数学运算。', 'parameters': {'pr
operties': {'op': {'type': 'string'}, 'a': {'type': 'number'}, 'b': {'type': 'num
ber'}}, 'required': ['op', 'a', 'b'], 'type': 'object'}}}]} config={} config_fact
ories=[]
```

In [48]:
```
response = model_with_tools.invoke("今天时间是几号?")
print(response)
print("*" * 30)
if response.tool_calls:
    print("AI 想调用工具: ", response.tool_calls)
else:
    print("AI 直接回答: ", response.content)
```

```
content='' additional_kwargs={} response_metadata={'model': 'qwen3.5:9b', 'create
d_at': '2026-04-07T13:38:11.082875Z', 'done': True, 'done_reason': 'stop', 'total
_duration': 4783548300, 'load_duration': 129511400, 'prompt_eval_count': 395, 'pr
ompt_eval_duration': 839294400, 'eval_count': 43, 'eval_duration': 3787709600, 'l
ogprobs': None, 'model_name': 'qwen3.5:9b', 'model_provider': 'ollama'} id='lc_ru
n--019d682a-2759-76f0-afe4-aa7c1ca2b33d-0' tool_calls=[{'name': 'get_current_tim
e', 'args': {}, 'id': 'b3b95c77-258f-4aaa-9386-ca23bff9d365', 'type': 'tool_cal
l'}] invalid_tool_calls=[] usage_metadata={'input_tokens': 395, 'output_tokens':
43, 'total_tokens': 438}
******************************
AI 想调用工具： [{'name': 'get_current_time', 'args': {}, 'id': 'b3b95c77-258f-4a
aa-9386-ca23bff9d365', 'type': 'tool_call'}]
```

In [49]:

```python
response = model_with_tools.invoke("使用win+d键，清空桌面.")
print(response)
print("*" * 30)
if response.tool_calls:
    print("AI 想调用工具：", response.tool_calls)
else:
    print("AI 直接回答：", response.content)
```

```
content='' additional_kwargs={} response_metadata={'model': 'qwen3.5:9b', 'create
d_at': '2026-04-07T13:38:21.8131207Z', 'done': True, 'done_reason': 'stop', 'tota
l_duration': 10718417400, 'load_duration': 167095200, 'prompt_eval_count': 398,
'prompt_eval_duration': 605345600, 'eval_count': 102, 'eval_duration': 988298720
0, 'logprobs': None, 'model_name': 'qwen3.5:9b', 'model_provider': 'ollama'} id
='lc_run--019d682a-3a15-7a93-8412-a0c0da2cc541-0' tool_calls=[{'name': 'control_s
ystem', 'args': {'op': 'win+d'}, 'id': 'e3b01491-727a-48df-a46a-27886c6b821f', 't
ype': 'tool_call'}] invalid_tool_calls=[] usage_metadata={'input_tokens': 398, 'o
utput_tokens': 102, 'total_tokens': 500}
******************************
AI 想调用工具： [{'name': 'control_system', 'args': {'op': 'win+d'}, 'id': 'e3b01
491-727a-48df-a46a-27886c6b821f', 'type': 'tool_call'}]
```

In [50]:

```python
response = model_with_tools.invoke("帮我计算23加40等于多少?")
print(response)
print("*" * 30)
if response.tool_calls:
    print("AI 想调用工具：", response.tool_calls)
else:
    print("AI 直接回答：", response.content)
```

```
content='' additional_kwargs={} response_metadata={'model': 'qwen3.5:9b', 'create
d_at': '2026-04-07T13:38:36.0997954Z', 'done': True, 'done_reason': 'stop', 'tota
l_duration': 14273570500, 'load_duration': 144980700, 'prompt_eval_count': 400,
'prompt_eval_duration': 673055800, 'eval_count': 140, 'eval_duration': 1338041390
0, 'logprobs': None, 'model_name': 'qwen3.5:9b', 'model_provider': 'ollama'} id
='lc_run--019d682a-63ff-7891-9b9c-e62b4b64247b-0' tool_calls=[{'name': 'calcuto
r', 'args': {'op': '+', 'a': 23, 'b': 40}, 'id': 'd0e00dd5-2902-4da7-845a-d0b7be8
32d37', 'type': 'tool_call'}] invalid_tool_calls=[] usage_metadata={'input_token
s': 400, 'output_tokens': 140, 'total_tokens': 540}
******************************
AI 想调用工具： [{'name': 'calcutor', 'args': {'op': '+', 'a': 23, 'b': 40}, 'i
d': 'd0e00dd5-2902-4da7-845a-d0b7be832d37', 'type': 'tool_call'}]
```

In [51]:

```python
response = model_with_tools.invoke("帮我计算23加40等于多少?运算符请使用汉字。")
print(response)
print("*" * 30)
if response.tool_calls:
    print("AI 想调用工具：", response.tool_calls)
```

```
else:
    print("AI 直接回答：", response.content)
```

```
content='' additional_kwargs={} response_metadata={'model': 'qwen3.5:9b', 'create
d_at': '2026-04-07T13:38:48.0524585Z', 'done': True, 'done_reason': 'stop', 'tota
l_duration': 11940257400, 'load_duration': 157513400, 'prompt_eval_count': 404,
'prompt_eval_duration': 855868900, 'eval_count': 112, 'eval_duration': 1086509940
0, 'logprobs': None, 'model_name': 'qwen3.5:9b', 'model_provider': 'ollama'} id
='lc_run--019d682a-9bcd-7f31-a4e1-6763aadd5e15-0' tool_calls=[{'name': 'calcuto
r', 'args': {'op': '加', 'a': 23, 'b': 40}, 'id': 'e387d234-0944-442b-b17b-0a3d9a
fa3b31', 'type': 'tool_call'}] invalid_tool_calls=[] usage_metadata={'input_token
s': 404, 'output_tokens': 112, 'total_tokens': 516}
******************************
AI 想调用工具： [{'name': 'calcutor', 'args': {'op': '加', 'a': 23, 'b': 40}, 'i
d': 'e387d234-0944-442b-b17b-0a3d9afa3b31', 'type': 'tool_call'}]
```

3.3. 设置工具属性

- 能让模型更好的理解工具(提升工具调用的智能)，还有两个属性可以设置；
  - name属性：默认情况下，工具名称源自函数名。当您需要更具描述性的名称时，可以通过name属性设置。
  - description属性：覆盖自动生成的工具描述，以便为模型提供更清晰的指引。

In [52]:
```
from langchain_core.tools import tool
@tool
def control_system(op:str)->str:
    """
    执行对操作系统的控制。
    """
    print(F"参数：op={op}")
    # 调用ctypes完成win + D键操作
    return "已经控制成功"
print(control_system.name)
print(control_system.description)
```

```
control_system
执行对操作系统的控制。
```

In [53]:
```
@tool("控制系统", description="控制系统，包括鼠标操作，快捷键按键操作，系统底层直
def control_system(op:str)->str:
    """
    执行对操作系统的控制。
    """
    print(F"参数：op={op}")
    # 调用ctypes完成win + D键操作
    return "已经控制成功"
print(control_system.name)
print(control_system.description)
```

```
控制系统
控制系统，包括鼠标操作，快捷键按键操作，系统底层直接调用
```

- 代码说明：
  - name属性必须是位置参数，不能使用关键字参数使用。

3.4. 工具的其他属性

- 工具还有其他属性，包含：
  - args_schema：用于验证和解析工具输入参数的 Pydantic 模型类。参数模式（args schema）可以是以下之一：
    - pydantic.BaseModel 的子类
    - 在 Pydantic 2 中访问 v1 命名空间时，pydantic.v1.BaseModel 的子类
    - 一个 JSON 模式字典
  - return_direct：是否直接返回工具的输出。将此参数设置为 True 意味着在工具被调用后，AgentExecutor 将停止循环。
  - verbose：是否记录工具的进度。
  - callbacks：在工具执行期间被调用的回调函数。
  - tags：与工具关联的可选标签列表。这些标签将与此工具的每次调用相关联，并作为参数传递给 callbacks 中定义的处理函数。可以使用它们来根据使用场景识别工具的特定实例。
  - metadata：与工具关联的可选元数据。此元数据将与此工具的每次调用相关联，并作为参数传递给 callbacks 中定义的处理函数。可以使用它们来，例如，根据使用场景识别工具的特定实例。
  - response_format: `Literal["content", "content_and_artifact"] = "content"` :工具响应格式。
    - 如果是 'content'，则工具的输出将被解释为 ToolMessage 的内容。
    - 如果是 'content_and_artifact'，则输出应为一个二元组，对应于 ToolMessage 的 (content, artifact)。
  - extras：工具的可选提供商特定额外字段。这用于传递不适合标准工具字段的、提供商特定的配置。

In [54]:
```
from langchain_core.tools import tool
@tool
def control_system(op:str)->str:
    """
    执行对操作系统的控制。
    """
    print(F"参数: op={op}")
    # 调用ctypes完成win + D键操作
    return "已经控制成功"
print(control_system.args_schema)
print(control_system.return_direct)
print(control_system.verbose)
print(control_system.callbacks)
print(control_system.tags)
print(control_system.metadata)
print(control_system.response_format)
print(control_system.extras)
```

```
<class 'langchain_core.utils.pydantic.control_system'>
False
False
None
None
None
content
None
```

(1) args_schema属性使用

In [55]:
```python
from pydantic import BaseModel, Field
from typing import Literal
from langchain_core.tools import tool

class ControlInput(BaseModel):
    """
    操作系统的控制输入
    """
    op: str = Field(description="对系统的操作类型，包括鼠标控制，键盘按键，清空桌
    # other: str = Field(default="缺省值", description="描述")

@tool(args_schema=ControlInput)
def control_system(op:str)->str:
    """
    执行对操作系统的控制。
    """
    print(F"参数：op={op}")
    # 调用ctypes完成win + D键操作
    return "已经控制成功"
print(control_system.name)
print(control_system.description)
print(control_system.args_schema)
```

```
control_system
执行对操作系统的控制。
<class '__main__.ControlInput'>
```

- 代码说明:
  - 使用args_schema来提供更加智能的参数模式。

4.5. 工具中的保留属性

- 工具中有两个属性不能使用，这两个属性是为未来保留的。
  - config：预留给内部向工具传递 RunnableConfig 使用。
  - runtime：预留给 ToolRuntime 参数（用于访问状态、上下文、存储）

- 当工具能够访问运行时信息（如对话历史、用户数据和持久化内存）时，它们的功能最为强大。如何在工具内部访问和更新这些信息。工具可以通过 ToolRuntime参数访问运行时信息，该参数提供：
  - State：短期记忆-存在于当前对话中的可变数据（消息、计数器、自定义字段），用于访问对话历史，追踪工具调用次数。

- Context：在调用时传递的不可变配置（用户ID、会话信息）。用于根据用户身份个性化响应。
- Store：长期记忆 - 跨对话持久化存储的数据。用于保存用户偏好，维护知识库。
- Stream Writer：在工具执行期间发出实时更新。用于显示长时间运行的操作的进度。
- Config：执行的 RunnableConfig。用于访问回调函数、标签和元数据。
- Tool Call ID：当前工具调用的唯一标识符。用于关联工具调用与日志及模型调用，用于追踪和调试。

In [56]:

```
from langchain.tools import tool, ToolRuntime
from langchain_core.runnables import RunnableConfig
from langchain.messages import HumanMessage
from langchain.agents import create_agent
from langchain.chat_models import init_chat_model
from langchain_core.tools import tool

@tool
def get_message(runtime: ToolRuntime, config:RunnableConfig) -> str:
    """访问用户最近的历史数据信息"""
    print(runtime)
    print("*" * 30)
    print(runtime.state)
    print("*" * 30)
    print(runtime.context)
    print("*" * 30)
    print(runtime.store)
    print("*" * 30)
    print(runtime.stream_writer)
    print("*" * 30)
    print(config)
    return "查询的用户信息与配置是50"

from langchain.chat_models import init_chat_model
model = init_chat_model(
    model="qwen3.5:9b",          # 你在 Ollama 中下载的模型名称
    model_provider="ollama",      # 关键参数，指定使用 ollama 提供商
    temperature=0.7,              # 其他参数与使用任何其他模型一样
    base_url="http://localhost:11434"  # 可选，如果你的 Ollama 不是默认地址，则需
)

# model_with_tools = model.bind_tools([get_message])
# response = model_with_tools.invoke("当前用户的访问数据量")
# print(response)

agent = create_agent(
    model=model,
    tools=[get_message],  # 只给一个工具
    system_prompt="你是一个数据助手，专门处理用户信息数据。"
)

response = agent.invoke({
    "messages": [{"role": "user", "content": "当前用户的访问数据量"}]
})

print(f"\nAgent 回复：{response['messages'][-1].content}")
```

```
ToolRuntime(state={'messages': [HumanMessage(content='当前用户的访问数据量', addi
tional_kwargs={}, response_metadata={}, id='5d25c42f-6c0f-4362-8805-89e1cac1516
d'), AIMessage(content='', additional_kwargs={}, response_metadata={'model': 'qwe
n3.5:9b', 'created_at': '2026-04-07T13:38:54.6089583Z', 'done': True, 'done_reaso
n': 'stop', 'total_duration': 5214497800, 'load_duration': 126094200, 'prompt_eva
l_count': 270, 'prompt_eval_duration': 623345800, 'eval_count': 47, 'eval_duratio
n': 4439557400, 'logprobs': None, 'model_name': 'qwen3.5:9b', 'model_provider':
'ollama'}, id='lc_run--019d682a-cfb0-73b1-926d-f4089d5860fb-0', tool_calls=[{'nam
e': 'get_message', 'args': {}, 'id': '901a7b70-e72e-447a-9e58-e49e7e090bdf', 'typ
e': 'tool_call'}], invalid_tool_calls=[], usage_metadata={'input_tokens': 270, 'o
utput_tokens': 47, 'total_tokens': 317})]}, context=None, config={'tags': [], 'me
tadata': {'langgraph_step': 2, 'langgraph_node': 'tools', 'langgraph_triggers':
('__pregel_push',), 'langgraph_path': ('__pregel_push', 0, False), 'langgraph_che
ckpoint_ns': 'tools:e7d9c1ad-eade-af90-81ee-18751c978ad6', 'checkpoint_ns': 'tool
s:e7d9c1ad-eade-af90-81ee-18751c978ad6'}, 'callbacks': <langchain_core.callbacks.
manager.CallbackManager object at 0x000001E08E426550>, 'recursion_limit': 25, 'co
nfigurable': {'__pregel_runtime': Runtime(context=None, store=None, stream_writer
=<function Pregel.stream.<locals>.stream_writer at 0x000001E0E1A8EAC0>, previous=
None), '__pregel_task_id': 'e7d9c1ad-eade-af90-81ee-18751c978ad6', '__pregel_sen
d': <built-in method extend of collections.deque object at 0x000001E08E4B6A70>,
'__pregel_read': functools.partial(<function local_read at 0x000001E08D02A5C0>, P
regelScratchpad(step=2, stop=10000, call_counter=<langgraph.pregel._algo.LazyAtom
icCounter object at 0x000001E08D3FB5B0>, interrupt_counter=<langgraph.pregel._alg
o.LazyAtomicCounter object at 0x000001E08D3FA080>, get_null_resume=<function _scr
atchpad.<locals>.get_null_resume at 0x000001E08D3BEE80>, resume=[], subgraph_coun
ter=<langgraph.pregel._algo.LazyAtomicCounter object at 0x000001E08D3FB9D0>), {'m
essages': <langgraph.channels.binop.BinaryOperatorAggregate object at 0x000001E08
E48DCC0>, 'jump_to': <langgraph.channels.ephemeral_value.EphemeralValue object at
0x000001E08E48D280>, 'structured_response': <langgraph.channels.last_value.LastVa
lue object at 0x000001E08E48D140>, '__start__': <langgraph.channels.ephemeral_val
ue.EphemeralValue object at 0x000001E08E48F980>, '__pregel_tasks': <langgraph.cha
nnels.topic.Topic object at 0x000001E08E48F280>, 'branch:to:model': <langgraph.ch
annels.ephemeral_value.EphemeralValue object at 0x000001E08E48E400>, 'branch:to:t
ools': <langgraph.channels.ephemeral_value.EphemeralValue object at 0x000001E08E4
8C180>}, {}, PregelTaskWrites(path=('__pregel_push', 0, False), name='tools', wri
tes=deque([]), triggers=('__pregel_push',))), '__pregel_checkpointer': None, 'che
ckpoint_map': {'': '1f132871-eee6-6d91-8001-f8b4ea0fcc23'}, 'checkpoint_id': Non
e, 'checkpoint_ns': 'tools:e7d9c1ad-eade-af90-81ee-18751c978ad6', '__pregel_scrat
chpad': PregelScratchpad(step=2, stop=10000, call_counter=<langgraph.pregel._alg
o.LazyAtomicCounter object at 0x000001E08D3FB5B0>, interrupt_counter=<langgraph.p
regel._algo.LazyAtomicCounter object at 0x000001E08D3FA080>, get_null_resume=<fun
ction _scratchpad.<locals>.get_null_resume at 0x000001E08D3BEE80>, resume=[], sub
graph_counter=<langgraph.pregel._algo.LazyAtomicCounter object at 0x000001E08D3FB
9D0>), '__pregel_call': functools.partial(<function _call at 0x000001E08D068540>,
<weakref at 0x000001E08E498F90; to 'langgraph.types.PregelExecutableTask' at 0x00
0001E08E3F6690>, retry_policy=None, futures=<weakref at 0x000001E08E499CB0; to 'l
anggraph.pregel._runner.FuturesDict' at 0x000001E08E498EB0>, schedule_task=<bound
method SyncPregelLoop.accept_push of <langgraph.pregel._loop.SyncPregelLoop objec
t at 0x000001E08D2C65D0>>, submit=<weakref at 0x000001E08E3EE4D0; to 'langgraph.p
regel._executor.BackgroundExecutor' at 0x000001E08D3AF5C0>)}}, stream_writer=<fun
ction Pregel.stream.<locals>.stream_writer at 0x000001E0E1A8EAC0>, tool_call_id
='901a7b70-e72e-447a-9e58-e49e7e090bdf', store=None)
******************************
{'messages': [HumanMessage(content='当前用户的访问数据量', additional_kwargs={},
response_metadata={}, id='5d25c42f-6c0f-4362-8805-89e1cac1516d'), AIMessage(conte
nt='', additional_kwargs={}, response_metadata={'model': 'qwen3.5:9b', 'created_a
t': '2026-04-07T13:38:54.6089583Z', 'done': True, 'done_reason': 'stop', 'total_d
uration': 5214497800, 'load_duration': 126094200, 'prompt_eval_count': 270, 'prom
pt_eval_duration': 623345800, 'eval_count': 47, 'eval_duration': 4439557400, 'log
probs': None, 'model_name': 'qwen3.5:9b', 'model_provider': 'ollama'}, id='lc_run
```

```
--019d682a-cfb0-73b1-926d-f4089d5860fb-0', tool_calls=[{'name': 'get_message', 'a
rgs': {}, 'id': '901a7b70-e72e-447a-9e58-e49e7e090bdf', 'type': 'tool_call'}], in
valid_tool_calls=[], usage_metadata={'input_tokens': 270, 'output_tokens': 47, 't
otal_tokens': 317})]}
******************************
None
******************************
None
******************************
<function Pregel.stream.<locals>.stream_writer at 0x000001E0E1A8EAC0>
******************************
{'tags': [], 'metadata': {'langgraph_step': 2, 'langgraph_node': 'tools', 'langgr
aph_triggers': ('__pregel_push',), 'langgraph_path': ('__pregel_push', 0, False),
'langgraph_checkpoint_ns': 'tools:e7d9c1ad-eade-af90-81ee-18751c978ad6', 'checkpo
int_ns': 'tools:e7d9c1ad-eade-af90-81ee-18751c978ad6'}, 'callbacks': <langchain_c
ore.callbacks.manager.CallbackManager object at 0x000001E08E5838D0>, 'recursion_l
imit': 25, 'configurable': {'__pregel_runtime': Runtime(context=None, store=None,
stream_writer=<function Pregel.stream.<locals>.stream_writer at 0x000001E0E1A8EAC
0>, previous=None), '__pregel_task_id': 'e7d9c1ad-eade-af90-81ee-18751c978ad6',
'__pregel_send': <built-in method extend of collections.deque object at 0x000001E
08E4B6A70>, '__pregel_read': functools.partial(<function local_read at 0x000001E0
8D02A5C0>, PregelScratchpad(step=2, stop=10000, call_counter=<langgraph.pregel._a
lgo.LazyAtomicCounter object at 0x000001E08D3FB5B0>, interrupt_counter=<langgrap
h.pregel._algo.LazyAtomicCounter object at 0x000001E08D3FA080>, get_null_resume=<
function _scratchpad.<locals>.get_null_resume at 0x000001E08D3BEE80>, resume=[],
subgraph_counter=<langgraph.pregel._algo.LazyAtomicCounter object at 0x000001E08D
3FB9D0>), {'messages': <langgraph.channels.binop.BinaryOperatorAggregate object a
t 0x000001E08E48DCC0>, 'jump_to': <langgraph.channels.ephemeral_value.EphemeralVa
lue object at 0x000001E08E48D280>, 'structured_response': <langgraph.channels.las
t_value.LastValue object at 0x000001E08E48D140>, '__start__': <langgraph.channel
s.ephemeral_value.EphemeralValue object at 0x000001E08E48F980>, '__pregel_tasks':
<langgraph.channels.topic.Topic object at 0x000001E08E48F280>, 'branch:to:model':
<langgraph.channels.ephemeral_value.EphemeralValue object at 0x000001E08E48E400>,
'branch:to:tools': <langgraph.channels.ephemeral_value.EphemeralValue object at 0
x000001E08E48C180>}, {}, PregelTaskWrites(path=('__pregel_push', 0, False), name
='tools', writes=deque([]), triggers=('__pregel_push',))), '__pregel_checkpointe
r': None, 'checkpoint_map': {'': '1f132871-eee6-6d91-8001-f8b4ea0fcc23'}, 'checkp
oint_id': None, 'checkpoint_ns': 'tools:e7d9c1ad-eade-af90-81ee-18751c978ad6', '_
_pregel_scratchpad': PregelScratchpad(step=2, stop=10000, call_counter=<langgrap
h.pregel._algo.LazyAtomicCounter object at 0x000001E08D3FB5B0>, interrupt_counter
=<langgraph.pregel._algo.LazyAtomicCounter object at 0x000001E08D3FA080>, get_nul
l_resume=<function _scratchpad.<locals>.get_null_resume at 0x000001E08D3BEE80>, r
esume=[], subgraph_counter=<langgraph.pregel._algo.LazyAtomicCounter object at 0x
000001E08D3FB9D0>), '__pregel_call': functools.partial(<function _call at 0x00000
1E08D068540>, <weakref at 0x000001E08E498F90; to 'langgraph.types.PregelExecutabl
eTask' at 0x000001E08E3F6690>, retry_policy=None, futures=<weakref at 0x000001E08
E499CB0; to 'langgraph.pregel._runner.FuturesDict' at 0x000001E08E498EB0>, schedu
le_task=<bound method SyncPregelLoop.accept_push of <langgraph.pregel._loop.SyncP
regelLoop object at 0x000001E08D2C65D0>>, submit=<weakref at 0x000001E08E3EE4D0;
to 'langgraph.pregel._executor.BackgroundExecutor' at 0x000001E08D3AF5C0>)}}
```

Agent 回复：根据查询结果，当前用户的访问数据量为 **50**。